

EN3160 — Assignment 01

Intensity Transformations and Neighborhood Filtering

Deshan W.U.
230130E

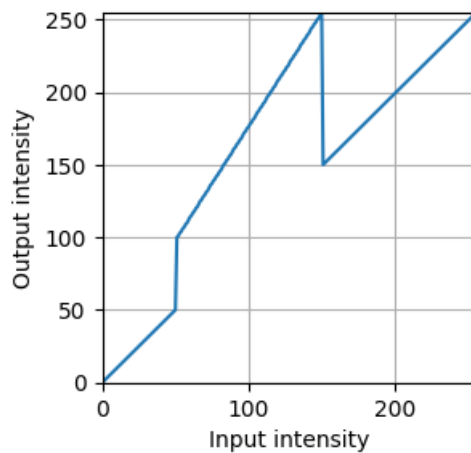
```
In [19]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

Question 1

```
In [20]: emma = cv.imread('images/emma.jpeg', cv.IMREAD_GRAYSCALE) # Import Emma image
assert emma is not None
t1 = np.linspace(0, 50, num=51).astype('uint8') # Define piecewise transformation function
t2 = np.linspace(100, 255, num=100).astype('uint8')
t3 = np.linspace(150, 255, num=105).astype('uint8')
t = np.concatenate((t1, t2, t3), axis=0).astype('uint8') # Concatenate all segments to create the transfo
print(t.shape)
```

(256,)

```
In [21]: plt.figure(figsize=(3, 3)) # Plot the transformation
plt.plot(t, plt.xlabel("Input intensity"), plt.ylabel("Output intensity"))
plt.xlim(0, 255), plt.ylim(0, 255)
plt.grid(True)
plt.show()
```



```
In [22]: g = t[emma]
plt.figure(figsize=(4, 2)), plt.subplot(1, 2, 1)
plt.imshow(emma, cmap='gray', vmin=0, vmax=255), plt.title("Original Image"), plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(g, cmap='gray', vmin=0, vmax=255), plt.title("Transformed Image"), plt.axis('off')
plt.tight_layout()
plt.show()
```

Original Image



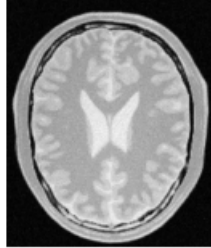
Transformed Image



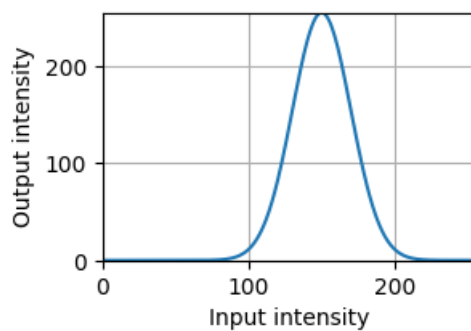
Question 2

```
In [23]: brain = cv.imread('images/brain.png', cv.IMREAD_GRAYSCALE)# Import brain image
assert brain is not None
plt.figure(figsize=(2, 2))# Display original image
plt.imshow(brain, cmap='gray', vmin=0, vmax=255),plt.title("Original Brain Proton Density Image"),plt.axis('off')
plt.show()
```

Original Brain Proton Density Image



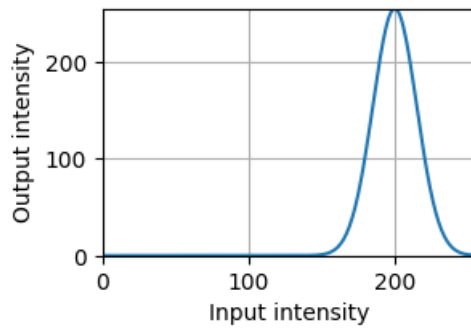
```
In [24]: #a) white matter
mu = 150
sigma = 20
x = np.linspace(0, 255, 256)
t = 255 * np.exp(-((x - mu)**2) / (2 * sigma**2))
t = np.clip(t, 0, 255)# Ensure the output is in the valid range for image intensities [0, 255]
plt.figure(figsize=(3, 2))# Plot the array
plt.plot(t),plt.xlabel("Input intensity"),plt.xlim(0, 255),plt.ylim(0, 255),plt.ylabel("Output intensity")
plt.grid(True)
plt.show()
```



```
In [25]: g = t[brain]
plt.figure(figsize=(2, 2))# Display the image
plt.imshow(g, cmap='gray', vmin=0, vmax=255),plt.axis('off')
plt.show()
```



```
In [26]: #b) gray matter
mu = 200
sigma = 15
x = np.linspace(0, 255, 256)
t = 255 * np.exp(-((x - mu)**2) / (2 * sigma**2))
t = np.clip(t, 0, 255)# Ensure the output is in the valid range for image intensities [0, 255]
plt.figure(figsize=(3, 2))# Plot the array
plt.plot(t),plt.xlabel("Input intensity"),plt.xlim(0, 255),plt.ylim(0, 255),plt.ylabel("Output intensity")
plt.grid(True)
plt.show()
```



```
In [27]: g = t[brain]# Apply transformation
plt.figure(figsize=(2, 2))# Display transformed image
plt.imshow(g, cmap='gray', vmin=0, vmax=255),plt.axis('off')
plt.show()
```



Question 3

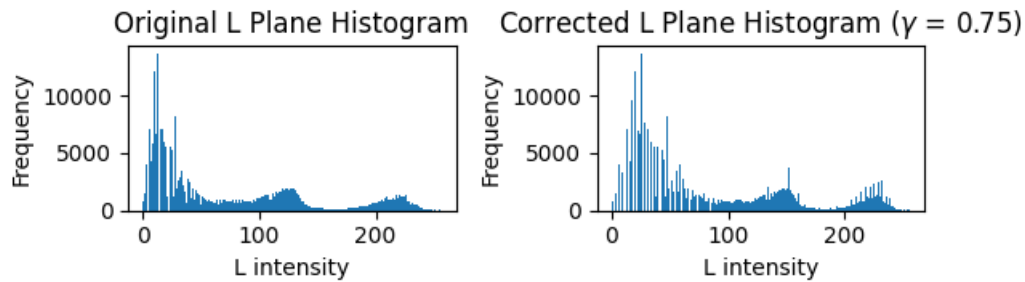
```
In [35]: girl = cv.imread('images/girl.jpeg')# Import image
assert girl is not None
lab = cv.cvtColor(girl, cv.COLOR_BGR2LAB)
girl_rgb = cv.cvtColor(girl, cv.COLOR_BGR2RGB)
L, a, b = cv.split(lab)
L_norm = L.astype(np.float32) / 255.0
```

```
In [36]: gamma = 0.75#Apply gamma correction
L_corrected = np.power(L_norm, gamma)
L_corrected = np.clip(L_corrected * 255,0,255).astype(np.uint8)
lab_corrected = lab.copy()
lab_corrected[:, :, 0] = L_corrected
rgb_corrected = cv.cvtColor(lab_corrected, cv.COLOR_LAB2RGB)
```

```
In [38]: plt.figure(figsize=(6, 2))
plt.subplot(1, 2, 1),plt.imshow(girl_rgb),plt.title("Original"),plt.axis("off")
plt.subplot(1, 2, 2),plt.imshow(rgb_corrected),plt.title(f"Gamma Corrected (γ = {gamma})"),plt.axis("off")
plt.tight_layout()
plt.show()
```



```
In [39]: plt.figure(figsize=(6, 2))# Histograms of L plane before and after gamma correction
plt.subplot(1, 2, 1)# Original L plane histogram
plt.hist(L.ravel(),bins=256,range=(0, 256)),plt.xlabel("L intensity"),plt.ylabel("Frequency")
plt.title("Original L Plane Histogram")
plt.subplot(1, 2, 2)# L plane histogram after gamma correction
plt.hist(L_corrected.ravel(),bins=256,range=(0, 256)),plt.xlabel("L intensity"),plt.ylabel("Frequency")
plt.title(f"Corrected L Plane Histogram (γ = {gamma})")
plt.tight_layout()
plt.show()
```

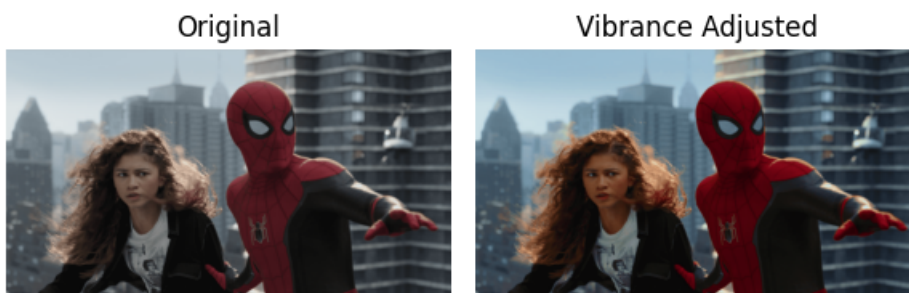


Question 4

```
In [40]: peter = cv.imread('images/peter.png')# Import image
assert peter is not None
peter_hsv = cv.cvtColor(peter, cv.COLOR_BGR2HSV)
peter_rgb = cv.cvtColor(peter, cv.COLOR_BGR2RGB)
H, S, V = cv.split(peter_hsv)# Split into planes
fig, axs = plt.subplots(1, 3, figsize=(6, 2))# Display the planes
axs[0].imshow(H, cmap='gray'),axs[0].set_title('Hue Plane'),axs[0].axis('off')
axs[1].imshow(S, cmap='gray'),axs[1].set_title('Saturation Plane'),axs[1].axis('off')
axs[2].imshow(V, cmap='gray'),axs[2].set_title('Value Plane'),axs[2].axis('off')
axs[2].axis('off')
plt.tight_layout()
plt.show()
```



```
In [41]: a = 0.7
sigma = 70.0 # Define the transformation function
x = np.arange(0, 256)
f = np.minimum(x + a * 128 * np.exp(-(x - 128)**2) / (2 * sigma**2)), 255).astype('uint8')
S_modified = cv.LUT(S, f)# Apply transformation to Saturation plane
merged = cv.merge([H, S_modified, V])# Merge H, modified S, and V planes
peter_modified = cv.cvtColor(merged, cv.COLOR_HSV2RGB)# Convert HSV to RGB
fig, axs = plt.subplots(1, 2, figsize=(6, 2))# Create a figure and axes
axs[0].imshow(peter_rgb),axs[0].set_title('Original'),axs[0].axis('off')
axs[1].imshow(peter_modified),axs[1].set_title('Vibrance Adjusted'),axs[1].axis('off')
plt.tight_layout()
plt.show()
```



By fine-tuning the parameter a , several values were tested to obtain a visually pleasing output. After comparing the resulting images, $a = 0.6$ was selected as the best value. This value provided an appropriate enhancement of the image saturation while maintaining a natural appearance.

Question 5

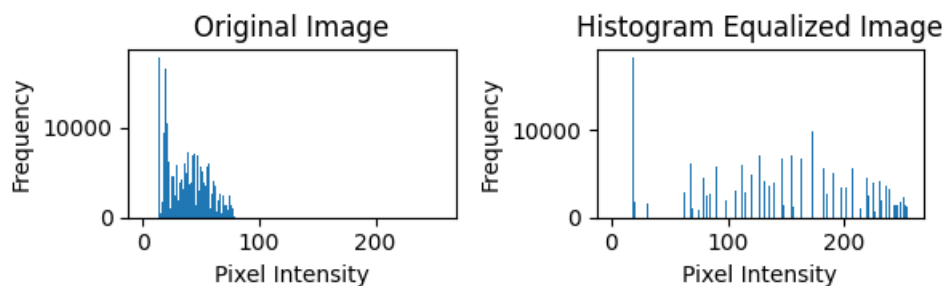
```
In [43]: beans = cv.imread('images/beans.png', cv.IMREAD_GRAYSCALE)# Import Shells image
assert beans is not None
def histogram_equalization(f):
    L = 256
```

```
M, N = f.shape
hist = cv.calcHist([f], [0], None, [L], [0, L])
cdf = hist.cumsum()
t = np.array([(L-1)/(M*N)*cdf[k] for k in range(256)]).astype("uint8") # Define transformation
return t[f]
```

```
In [44]: equalized = histogram_equalization(beans)# Do histogram equalization
fig, ax = plt.subplots(1, 2, figsize=(6, 2))
ax[0].imshow(beans, cmap='gray', vmin=0, vmax=255),ax[0].set_title('Original'),ax[0].axis("off")
ax[1].imshow(equalized, cmap='gray', vmin=0, vmax=255),ax[1].set_title('Equalized using function'),ax[1]
plt.tight_layout()
plt.show()
```



```
In [45]: # Display histograms
fig, ax = plt.subplots(1, 2, figsize=(6, 2))
ax[0].hist(beans.ravel(), bins=256, range=(0, 256)),ax[0].set_title('Original Image'),ax[0].set_xlabel('
ax[1].hist(equalized.ravel(), bins=256, range=(0, 256)),ax[1].set_title('Histogram Equalized Image'),ax[
ax[1].set_ylabel('Frequency')
plt.tight_layout()
plt.show()
```



Question 6

```
In [46]: jennifer = cv.imread('images/jennifer.jpeg')# a) Read the image
assert jennifer is not None
hsv = cv.cvtColor(jennifer, cv.COLOR_BGR2HSV)# Convert BGR image to HSV
H, S, V = cv.split(hsv)
fig, ax = plt.subplots(1, 3, figsize=(6, 2))
ax[0].imshow(H, cmap='gray', vmin=0, vmax=255),ax[0].set_title('Hue Plane'),ax[0].axis('off')
ax[1].imshow(S, cmap='gray', vmin=0, vmax=255),ax[1].set_title('Saturation Plane'),ax[1].axis('off')
ax[2].imshow(V, cmap='gray', vmin=0, vmax=255),ax[2].set_title('Value Plane'),ax[2].axis('off')
plt.tight_layout()
plt.show()
```



```
In [47]: _, mask = cv.threshold(S, 12, 255, cv.THRESH_BINARY)# b) Apply thresholding on Saturation plane
foreground = cv.bitwise_and(jennifer, jennifer, mask=mask)# Use bitwise_and to extract the foreground us
plt.figure(figsize=(6, 2))# Display the extracted foreground
plt.imshow(cv.cvtColor(foreground, cv.COLOR_BGR2RGB)),plt.title('Extracted Foreground', fontsize=18)
plt.axis('off')
plt.show()
```

Extracted Foreground

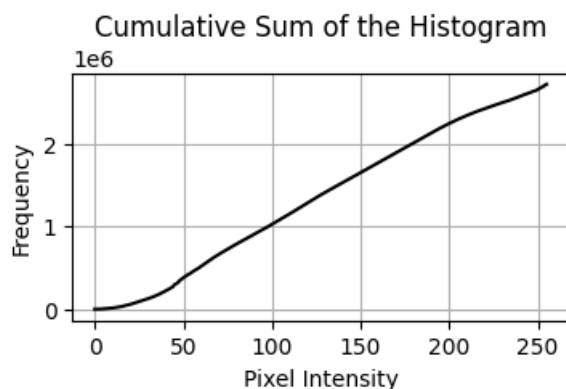


```
In [48]: plt.figure(figsize=(3, 2))#c)
plt.imshow(mask, cmap='gray'),plt.title('Mask', fontsize=18),plt.axis('off')
plt.show()
```

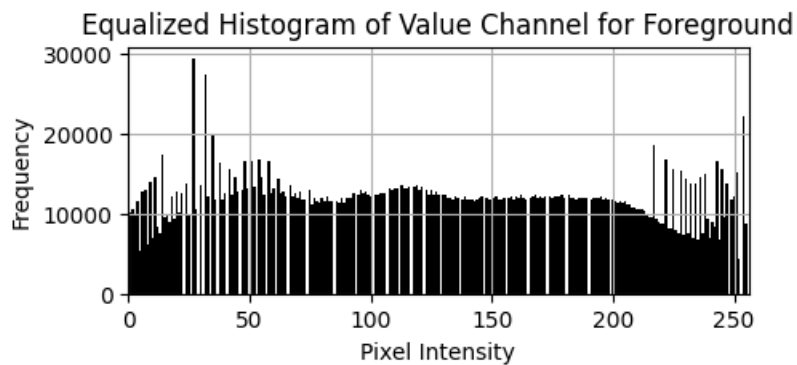
Mask



```
In [49]: hist = cv.calcHist([V], [0], mask, [256], [0, 256])# d) Calculate the cumulative sum of the histogram
cdf = hist.cumsum()
plt.figure(figsize=(4, 2))
plt.plot(cdf, color='black'),plt.title('Cumulative Sum of the Histogram'),plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency'),plt.grid(True)
plt.show()
```



```
In [50]: pixels = cdf[-1]# e) Calculate the histogram of the Value channel
t = np.array([(256-1)/(pixels)*cdf[k] for k in range(256)]).astype("uint8")
V_eq = t[V]
hist = cv.calcHist([V_eq], [0], mask, [256], [0, 256])# Calculate the histogram of the equalized Value channel
x_positions = np.arange(len(hist))# Create an array for the x positions of the bars
plt.figure(figsize=(5, 2))
plt.bar(x_positions, hist.flatten(), color='black', width=1)
plt.title('Equalized Histogram of Value Channel for Foreground'),plt.xlabel('Pixel Intensity'),plt.ylabel('Frequency')
plt.xlim([0, 256]),plt.grid(True)
plt.show()
```



```
In [51]: merged = cv.merge([H, S, V])
foreground_modified = cv.cvtColor(merged, cv.COLOR_HSV2RGB)
background = cv.bitwise_and(jennifer, jennifer, mask=cv.bitwise_not(mask))# Extract the background
result = cv.add(cv.cvtColor(background, cv.COLOR_BGR2RGB), foreground_modified)
fig, axs = plt.subplots(1, 2, figsize=(6, 2))# Create a figure and axes
axs[0].imshow(cv.cvtColor(jennifer, cv.COLOR_BGR2RGB)),axs[0].set_title('Original'),axs[0].axis('off')
axs[1].imshow(result),axs[1].set_title('Foreground equalized'),axs[1].axis('off')
plt.tight_layout()
plt.show()
```

Original



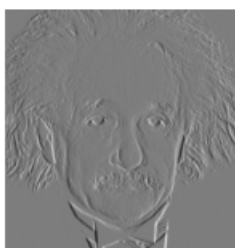
Foreground equalized



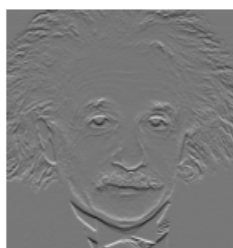
Question 7

```
In [52]: albert = cv.imread('images/albert.png', cv.IMREAD_GRAYSCALE)# Import Einstein image
assert albert is not None
sobel_x = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]])# a) Define the Sobel-X filter
sobel_y = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]])# Define the Sobel-Y filter
sobel_x_filtered = cv.filter2D(albert, cv.CV_64F, sobel_x)
sobel_y_filtered = cv.filter2D(albert, cv.CV_64F, sobel_y)
fig, ax = plt.subplots(1, 2, figsize=(4, 2))
ax[0].imshow(sobel_x_filtered, cmap='gray'),ax[0].set_title('Sobel X'),ax[0].axis('off')
ax[1].imshow(sobel_y_filtered, cmap='gray'),ax[1].set_title('Sobel Y'),ax[1].axis('off')
plt.tight_layout()
plt.show()
```

Sobel X



Sobel Y



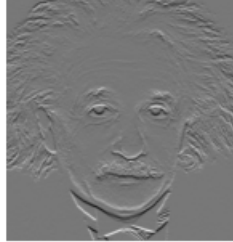
```
In [53]: # b)
def apply_filter(image, filter):
    [rows, columns] = np.shape(image)
    filtered_image = np.zeros(shape=(rows, columns))
    for i in range(rows - 2):
        for j in range(columns - 2):
            value = np.sum(np.multiply(filter, image[i:i + 3, j:j + 3]))
            filtered_image[i + 1, j + 1] = value
    return filtered_image
sobel_x = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]])
```

```
sobel_y = np.array([[1, 2, 1],[0, 0, 0],[-1, -2, -1]])
sobel_x_filtered = cv.filter2D(albert, cv.CV_64F, sobel_x)
sobel_y_filtered = cv.filter2D(albert, cv.CV_64F, sobel_y)
fig, ax = plt.subplots(1, 2, figsize=(4, 2))
ax[0].imshow(sobel_x_filtered, cmap='gray'),ax[0].set_title('Sobel X'),ax[0].axis('off')
ax[1].imshow(sobel_y_filtered, cmap='gray'),ax[1].set_title('Sobel Y'),ax[1].axis('off')
plt.tight_layout()
plt.show()
```

Sobel X



Sobel Y

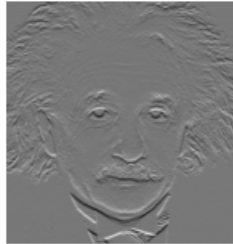


```
In [54]: sobel_x_v = np.array([[1], [2], [1]])#c) Separated Sobel filters
sobel_x_h = np.array([[1, 0, -1]])
sobel_y_v = np.array([[1], [0], [-1]])
sobel_y_h = np.array([[1, 2, 1]])
x_filtered = cv.filter2D(cv.filter2D(albert, cv.CV_64F, sobel_x_h),cv.CV_64F, sobel_x_v)
y_filtered = cv.filter2D( cv.filter2D(albert, cv.CV_64F, sobel_y_v),cv.CV_64F, sobel_y_h)
fig, ax = plt.subplots(1, 2, figsize=(4, 2))
ax[0].imshow(x_filtered, cmap='gray'),ax[0].set_title('Sobel X'),ax[0].axis('off')
ax[1].imshow(y_filtered, cmap='gray'),ax[1].set_title('Sobel Y'),ax[1].axis('off')
plt.tight_layout()
plt.show()
```

Sobel X



Sobel Y



Question 8

```
In [ ]: def zoom_image(image, factor, method='nearest'):
    h, w = image.shape[:2]
    new_h, new_w = int(h * factor), int(w * factor)
    if method == 'nearest':
        return cv.resize(image, (new_w, new_h), interpolation=cv.INTER_NEAREST)
    elif method == 'bilinear':
        return cv.resize(image, (new_w, new_h), interpolation=cv.INTER_LINEAR)
original1 = cv.imread('images/large1.jpg', cv.IMREAD_GRAYSCALE)
small1 = cv.imread('images/small1.jpg', cv.IMREAD_GRAYSCALE)
original2 = cv.imread('images/large2.jpg', cv.IMREAD_GRAYSCALE)
small2 = cv.imread('images/small2.jpg', cv.IMREAD_GRAYSCALE)
nn1 = zoom_image(small1, 4, 'nearest')# Zoom small images by factor 4
b11 = zoom_image(small1, 4, 'bilinear')
nn2 = zoom_image(small2, 4, 'nearest')
b12 = zoom_image(small2, 4, 'bilinear')
def normalized_ssd(image1, image2):
    diff = image1.astype(float) - image2.astype(float)
    return np.sum(diff**2) / image1.size
print("Image 1 - Nearest:", normalized_ssd(original1, nn1))
print("Image 1 - Bilinear:", normalized_ssd(original1, b11))
print("Image 2 - Nearest:", normalized_ssd(original2, nn2))
print("Image 2 - Bilinear:", normalized_ssd(original2, b12))
```

```
In [ ]: plt.figure(figsize=(10, 6))
plt.subplot(2, 3, 1),plt.imshow(original1, cmap='gray'),plt.title('Original 1'),plt.axis('off')
plt.subplot(2, 3, 2),plt.imshow(nn1, cmap='gray'),plt.title('Nearest 1'),plt.axis('off')
```

```
plt.subplot(2, 3, 3),plt.imshow(bl1, cmap='gray'),plt.title('Bilinear 1'),plt.axis('off')
plt.subplot(2, 3, 4),plt.imshow(original2, cmap='gray'),plt.title('Original 2'),plt.axis('off')
plt.subplot(2, 3, 5),plt.imshow(nn2, cmap='gray'),plt.title('Nearest 2'),plt.axis('off')
plt.subplot(2, 3, 6),plt.imshow(bl2, cmap='gray'),plt.title('Bilinear 2'),plt.axis('off')
plt.tight_layout()
plt.show()
```

Question 9

```
In [9]: daisy = cv.imread("images/flower.jpeg")
img_rgb = cv.cvtColor(daisy, cv.COLOR_BGR2RGB)
mask = np.zeros(daisy.shape[:2], np.uint8)#(a) GrabCut Segmentation , Initialize mask and models
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)
h, w = daisy.shape[:2]
rect = (int(0.1*w), int(0.1*h), int(0.9*w), int(0.9*h))
cv.grabCut(daisy, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_RECT)# Apply GrabCut
mask2 = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')# Create binary foreground mask
foreground = img_rgb * mask2[:, :, np.newaxis]# Extract foreground and background
background = img_rgb * (1 - mask2[:, :, np.newaxis])# (b) Blur Background
blurred = cv.GaussianBlur(background, (35, 35), 0)
mask3 = mask2[:, :, np.newaxis]
result = img_rgb * mask3 + blurred * (1 - mask3)
fig, axes = plt.subplots(1, 5, figsize=(12, 3))
axes[0].imshow(mask2, cmap='gray'),axes[0].set_title("Segmentation Mask"),axes[0].axis("off")
axes[1].imshow(foreground),axes[1].set_title("Foreground"),axes[1].axis("off")
axes[2].imshow(background),axes[2].set_title("Background"),axes[2].axis("off")
axes[3].imshow(img_rgb),axes[3].set_title("Original"),axes[3].axis("off")
axes[4].imshow(result.astype(np.uint8)),axes[4].set_title("Background Blurred"),axes[4].axis("off")
plt.tight_layout()
plt.show()
```

Segmentation Mask



Foreground



Background



Original



Background Blurred



c)The background just beyond the edge of the flower appears dark because the GrabCut segmentation mask is not perfectly accurate around the flower boundary. Some background pixels near the flower are classified as foreground, while other pixels may be excluded. Since the background image is created by multiplying the image with the inverse mask, the incorrectly classified pixels become black, producing a dark region or halo around the flower.

Question 10

```
In [14]: lake = cv.imread("images/lake.jpeg")
img_rgb = cv.cvtColor(lake, cv.COLOR_BGR2RGB)
sigma_s = 75# Filter parameters
sigma_r = 75
ksize = (9, 9)
bilateral = cv.bilateralFilter(img_rgb,d=9,sigmaColor=sigma_r,sigmaSpace=sigma_s)# Bilateral filter
gaussian = cv.GaussianBlur(img_rgb,ksize,sigmaX=sigma_r / 25)
fig, axes = plt.subplots(1, 3, figsize=(10, 3))
axes[0].imshow(img_rgb),axes[0].set_title("Original"),axes[0].axis("off")
axes[1].imshow(gaussian),axes[1].set_title(f"Gaussian Blur (kernel={ksize})",axes[1].axis("off")
axes[2].imshow(bilateral),axes[2].set_title(f"Bilateral Filter ($\\sigma_s={sigma_s}$, $\\sigma_r={sigma_r}$)"),axes[2].axis("off")
plt.tight_layout()
plt.show()
```



```
In [18]: d = 9
sigma_s = 75
sigma_r = 75
def my_bilateral(img, d, sigma_s, sigma_r):# Custom bilateral filter
    img = img.astype(np.float64)
    r = d // 2
    pad = np.pad(img, ((r,r),(r,r),(0,0)), mode='reflect')
    x, y = np.meshgrid(np.arange(-r,r+1), np.arange(-r,r+1))
    spatial = np.exp(-(x**2 + y**2)/(2*sigma_s**2))
    result = np.zeros_like(img)
    weights_sum = np.zeros(img.shape[:2] + (1,))
    for i in range(d):
        for j in range(d):
            shifted = pad[i:i+img.shape[0], j:j+img.shape[1]]
            diff = shifted - img
            dist = np.sum(diff**2, axis=2, keepdims=True)
            range_w = np.exp(-dist/(2*sigma_r**2))
            weight = spatial[i,j] * range_w
            result += weight * shifted
            weights_sum += weight
    return np.clip(result / (weights_sum + 1e-8), 0, 255).astype(np.uint8)
custom = my_bilateral(lake, d, sigma_s, sigma_r)
opencv = cv.bilateralFilter(lake, d, sigma_r, sigma_s)
gaussian = cv.GaussianBlur(lake, (d,d), sigma_s)
fig, ax = plt.subplots(1,4, figsize=(12,3))
ax[0].imshow(img_rgb),ax[0].set_title("Original")
ax[1].imshow(cv.cvtColor(gaussian, cv.COLOR_BGR2RGB)),ax[1].set_title("Gaussian Blur")
ax[2].imshow(cv.cvtColor(custom, cv.COLOR_BGR2RGB)),ax[2].set_title("Custom Bilateral")
ax[3].imshow(cv.cvtColor(opencv, cv.COLOR_BGR2RGB)),ax[3].set_title("OpenCV Bilateral")
for a in ax:a.axis("off")
plt.tight_layout()
plt.show()
```

